

Gold Store ERP – Enterprise System Design Prompt

Act as a Principal Software Architect, Enterprise Solution Architect, Domain-Driven Design Expert, ASP.NET Architect, Database Architect, ERP Consultant, and Technical Lead.

Design a complete production-ready System Design for a Gold Store ERP system.

The output must be implementation-ready and detailed enough for a senior engineering team to start development immediately.

Avoid generic explanations.

Provide architecture decisions with reasoning.

The solution must be suitable for real-world production use.

Architecture Constraints

The system **MUST** be designed as a:

Modular Monolith

Requirements:

- Single deployable application
- Clear module boundaries
- Independent domain modules
- Loose coupling between modules
- High cohesion inside modules
- Future migration to microservices should be possible without major redesign

DO NOT design Microservices.

DO NOT design a distributed system.

Project Overview

The system is a web-based ERP platform for managing a gold jewelry store.

The focus is:

- Gold inventory tracking
- Supplier management
- Gold purchases from customers
- Gold sales
- Supplier settlements
- Manufacturing fees
- Cash management
- Bank account management
- Expense management
- Salary management
- Gold balance tracking
- Financial balance tracking
- Reporting

This is NOT a full accounting ERP.

The objective is operational management and balance tracking.

Current scope:

- Single branch
 - Single company
-

Technology Stack

Backend

- ASP.NET Core 10
- C#
- Entity Framework Core
- PostgreSQL

Frontend

- Angular
- TypeScript
- Tailwind CSS

Authentication

- Email
- Password

Infrastructure

- Docker
- Ubuntu Linux
- Nginx Reverse Proxy

Cloud-ready architecture required.

Core Business Rules

Gold Capital

The store owns gold assets.

Supported karats:

- 18K
- 21K
- 24K

All balances and reports must normalize gold into:

21K Equivalent Weight

Formula:

$\text{Equivalent21Weight} = \text{Weight} \times (\text{Karat} / 21)$

Examples:

18K: $100\text{g} \times (18 / 21)$

24K: $100\text{g} \times (24 / 21)$

The 21K equivalent is the official reporting standard.

Cash Capital

Supported currencies:

- JOD
- USD
- ILS

Each currency may exist in:

Cash Accounts

Bank Accounts

The system must support unlimited accounts.

Functional Modules

Identity Module

Manage:

- Users
- Roles
- Permissions

Authentication:

- Email
- Password

User Profile:

- Display Name
- Phone Number
- Email

Default Roles:

- Admin
- Accountant
- Seller

Permissions must be configurable.

RBAC required.

Supplier Module

Supplier fields:

- Name
- Primary Phone
- Secondary Phone
- Bank Account Number
- Notes

Supplier balances consist of:

Gold Balance

Supplier Delivered Gold

minus

Scrap Gold Returned

Result:

Outstanding Gold Balance

Tracked in 21K equivalent.

Manufacturing Balance

Manufacturing Fees Owed

minus

Manufacturing Payments

Result:

Outstanding Manufacturing Balance

Category Module

Hierarchical categories.

Examples:

Rings

- Men
- Women

Bracelets

- Women
- Children

Necklaces

Coins

Other

Unlimited hierarchy depth.

Sales Module

Sales Invoice Header

- Invoice Number
- Date
- Customer Name
- Customer Bank Account (Optional)
- Seller
- Notes

Sales Invoice Lines

- Category
- Weight
- Karat
- Gram Price (JOD)
- Currency
- Total Amount

Support multiple invoice lines.

Sales Payments

Support multiple payments per invoice.

Fields:

- Payment Method
- Account
- Currency
- Amount

Payment Methods:

- Cash
- Bank Transfer

Currencies:

- JOD
- USD
- ILS

Customer Gold Purchase Module

Purchase Header

- Customer Name
- National ID
- Phone
- Address
- Birth Year
- Bank Account (Optional)
- Buyer
- Notes

Purchase Lines

- Category
- Weight
- Karat
- Gram Price
- Currency
- Total Price

Support multiple lines.

Payment

Source Account:

- Cash Account

- Bank Account
-

Supplier Delivery Module

Supplier delivers:

- Weight
- Karat

System stores:

- Weight
- Equivalent21Weight

Manufacturing:

- Fee Per Gram
- Total Fee

Balances must update automatically.

Supplier Payment Module

Scrap Gold Payment

Fields:

- Weight
- Karat
- Equivalent21Weight

Manufacturing Payment

Fields:

- Amount
 - Currency
 - Account
-

Inventory Module

Inventory must be movement-based.

Direct stock editing is prohibited.

Stock Increase Sources:

- Supplier Deliveries
- Customer Gold Purchases
- Positive Adjustments

Stock Decrease Sources:

- Sales
- Supplier Scrap Payments
- Negative Adjustments

Inventory tracked by:

- Weight
- Karat

Inventory Adjustment Module

Support:

- Increase
- Decrease
- Damage
- Loss
- Correction

Fields:

- Date
- Weight
- Karat
- Reason
- User
- Notes

Finance Module

Manage:

- Cash Accounts
- Bank Accounts

Fields:

- Name
- Currency
- Account Type

IMPORTANT:

Balances must NOT be stored directly.

Balances must be calculated from financial transactions.

Expense Module

Fields:

- Date
- Category
- Description
- Amount
- Currency
- Account
- User

Categories:

- Rent
 - Electricity
 - Water
 - Internet
 - Transportation
 - Maintenance
 - Miscellaneous
-

HR Module

Employee:

- Name
- Phone
- Position
- Salary

Salary Payment:

- Employee
 - Month
 - Amount
 - Currency
 - Account
 - Date
-

Gold Price Module

Store historical gold prices.

Fields:

- Date
- 21K Gram Price
- Currency
- Created By

Used for:

- Reporting
 - Historical reference
 - Trends
-

Ledger Architecture

Financial Ledger

All money movements must be recorded in a unified ledger.

Sources:

- Sales Payments
- Customer Gold Purchases
- Supplier Manufacturing Payments
- Expenses
- Salaries
- Manual Financial Adjustments

Fields:

- Date
- Account
- Currency
- Amount
- Transaction Type
- Reference Type
- Reference Id
- User

Account balances must be calculated from this ledger.

Gold Ledger

All gold movements must be recorded in a unified gold ledger.

Sources:

- Supplier Deliveries
- Customer Gold Purchases
- Sales
- Supplier Scrap Payments
- Inventory Adjustments

Fields:

- Date
- Karat
- Weight
- Equivalent21Weight
- Movement Type
- Reference Type
- Reference Id
- User

Inventory balances must be calculated from this ledger.

Required Output

Generate the following sections.

Section 1

Domain Analysis

Identify:

- Core Domains
- Supporting Domains
- Generic Domains

Apply Domain-Driven Design.

Identify Bounded Contexts.

Provide Context Map.

Section 2

High-Level Architecture

Provide:

Context Diagram

Container Diagram

Component Diagram

Module Dependency Diagram

Explain architectural decisions.

Section 3

Database Design

Provide complete:

- ERD
- Tables
- Columns
- Data Types
- PKs
- FKs

- Unique Constraints
- Check Constraints
- Indexes

Include PostgreSQL recommendations.

Explain normalization decisions.

Target 3NF minimum.

Section 4

Domain Model

Design:

- Aggregates
- Aggregate Roots
- Entities
- Value Objects
- Domain Services
- Domain Events

Explain invariants and business rules.

Section 5

Application Architecture

Use:

Clean Architecture

Layers:

- Domain
- Application
- Infrastructure
- Presentation

Provide:

- Folder Structure
- Project Structure

- Module Structure
-

Section 6

REST API Design

Design APIs for all modules.

Provide:

- Endpoint List
- Request DTOs
- Response DTOs
- Pagination
- Sorting
- Filtering
- Validation Rules

Use REST best practices.

Section 7

Security Architecture

Design:

Authentication

- Email/Password
- Argon2id Password Hashing
- JWT Access Tokens
- Refresh Token Rotation

Authorization

- RBAC
- Permission-based authorization

Security Features

- Audit Logging
- Account Lockout
- Password Reset
- Session Management

Section 8

Inventory Calculation Engine

Design algorithms for:

- Current Stock
- 21K Equivalent
- Supplier Gold Balance

Provide formulas and examples.

Section 9

Financial Engine

Design algorithms for:

- Cash Balance
- Bank Balance
- Multi-Currency Transactions

Provide formulas and examples.

Section 10

Reporting Architecture

Design reporting infrastructure.

Support:

- Dashboard Analytics
- PDF Export
- Excel Export

Optimize for large datasets.

Section 11

Performance & Scalability

Target:

50,000+ Transactions

Cover:

- Query Optimization
 - PostgreSQL Indexing Strategy
 - Caching
 - Background Jobs
 - Materialized Views
 - Read Models
-

Section 12

Deployment Architecture

Design:

- Docker Architecture
- Nginx Configuration
- PostgreSQL Deployment
- Backup Strategy
- Logging Strategy
- Monitoring Strategy

Target Ubuntu Linux production environment.

Section 13

Development Roadmap

Phase 1: Foundation & Identity

Phase 2: Inventory & Suppliers

Phase 3: Sales & Customer Gold Purchases

Phase 4: Finance & Reporting

Phase 5: Optimization & Hardening

For each phase provide:

- Objectives
- Deliverables
- Complexity
- Risks
- Priority

The final output must be a production-grade architecture document suitable for direct implementation by a professional software engineering team.